

Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **Digital Wallet System**.

In this article, we'll guide you through a clear step-by-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- The system should allow users to create wallets with unique account numbers.
- The system should enable transfer of funds between wallets using TUF currency.
- The system should allow users to add money to their wallets through payment gateway integration.
- The system should enforce minimum balance constraint - account balance cannot drop below TUF 0.00.
- The system should enforce minimum transfer amount constraint - smallest transferable amount is 0.01 TUF.
- Users should be able to view account statements showing all transactions for their wallet.
- The system should maintain a complete audit trail of all transactions.
- The system should handle concurrent transactions and ensure data consistency.

- Send notifications to email.
- Admin can suspend, reopen and close wallets.

Interview Tip: Always ask the interviewer to confirm the requirements before jumping into code. It shows clarity and alignment.

Counter Questions to Ask Interviewer

- How to handle concurrent transactions on the same wallet?
 - Which locking mechanism to use?
 - OTP confirmation on transactions?
 - Creating accounts has any authentication?
 - Maintain transaction limits?
 - Withdraw options to external source?
-

Step 2: Identify Core Entities

1. Wallet (Core Entity)

- id: int [PK]
- accountNumber: String [UNIQUE]
- balance: long (Stored as integer: actual amount * 100, e.g., 100.50 TUF = 10050)
- userId: int [FK to User, UNIQUE] (One-to-One: One user has one wallet)
- status: WalletStatus (Enum: ACTIVE, SUSPENDED, CLOSED)
- createdAt: LocalDateTime
- updatedAt: LocalDateTime

2. User

- id: int [PK]
- username: String [UNIQUE]
- email: String [UNIQUE]
- name: String
- (One-to-One relationship: One user has one wallet, accessed via Wallet.userId)

3. Transaction (Core Entity)

- id: int [PK]
- transactionId: String [UNIQUE]
- fromWalletId: int [FK to Wallet, NULLABLE] (NULL for DEPOSIT)
- toWalletId: int [FK to Wallet, NULLABLE] (NULL for WITHDRAWAL)
- amount: long (Stored as integer: actual amount * 100)
- transactionType: TransactionType (Enum: TRANSFER, DEPOSIT, WITHDRAWAL)
- status: TransactionStatus (Enum: PENDING, COMPLETED, FAILED, CANCELLED)
- paymentGatewayId: String [NULLABLE]
- paymentMethod: String [NULLABLE]
- description: String
- timestamp: LocalDateTime

4. TransactionStatus (Enum)

- PENDING
- COMPLETED
- FAILED
- CANCELLED

5. TransactionType (Enum)

- TRANSFER
- DEPOSIT
- WITHDRAWAL

6. WalletStatus (Enum)

- ACTIVE
- SUSPENDED
- CLOSED

7. AccountStatement (Response DTO – Not Stored)

- walletId: int
- walletAccountNumber: String
- transactions: List<Transaction>
- startDate: LocalDateTime (Optional)
- endDate: LocalDateTime (Optional)
- currentBalance: long
- Note: This is a generated response, not a stored entity.

8. TransactionRequest (Request DTO)

- fromAccountNumber: String
- toAccountNumber: String
- amount: long (actual amount * 100)
- description: String

9. AddMoneyRequest (Request DTO)

- accountNumber: String
- amount: long (actual amount * 100)
- paymentMethod: String
- paymentGateway: String
- paymentDetails: Map<String, String>

Interview Tip: Defining clean entities early helps ensure accurate handling of transfers, wallet balances, transaction auditing, and concurrency.

Step 3: Visualize Interaction Flows

1. Wallet Creation:

- POST /api/wallets → WalletController.createWallet(userId)
- → WalletService.createWallet(userId)
- → Validate one-wallet-per-user
- → Create Wallet (status=ACTIVE, balance=0)
- → Return Wallet

2. Add Money (Deposit) – Two Phase:

Phase 1 - Initiate Deposit:

- POST /api/wallets/{accountNumber}/deposit → TransactionController.initiateDeposit(request)
- → TransactionService.initiateDeposit(request)
- → Create Transaction(PENDING, type=DEPOSIT)
- → PaymentGatewayRouter.selectProvider(request.paymentGateway, amount, "TUF")
- → provider.initiatePayment(...)
- → Return providerRef to client

Phase 2 - Payment Callback:

- POST /api/payments/callback → TransactionController.handlePaymentCallback(providerRef, status)
- → TransactionService.handleDepositCallback(providerRef, status)
- → Verify signature
- → On SUCCESS: credit wallet, mark transaction COMPLETED → Notify user
- → On FAILURE/TIMEOUT: mark transaction FAILED → No balance change

3. Transfer Funds:

- POST /api/transactions/transfer → TransactionController.transfer(request)
- → TransactionService.transfer(request)
- → Acquire locks on both wallets (see STEP-7.5)
- → Validate wallets active + amount $\geq$ min + sufficient balance
- → Debit source → Credit destination
- → Create Transaction(COMPLETED, type=TRANSFER)
- → Notify user

4. Withdraw:

- POST /api/wallets/{accountNumber}/withdraw → TransactionController.withdraw(accountNumber, amount)
- → TransactionService.withdraw(...)
- → Validate limits
- → Create Transaction(PENDING, type=WITHDRAWAL)
- → External payout integration (out of scope)
- → On success: debit wallet and mark COMPLETED; else FAILED

5. Account Statement:

- GET /api/wallets/{accountNumber}/statement?start&end → WalletController.getStatement(...)
- → TransactionService.getStatement(accountNumber, start, end)
- → Build AccountStatement
- → Return

6. Notifications:

- On transaction completion → NotificationRouter.send("email", NotificationMessage{to, subject, body})

Step 4: Defines Class Structures & Relationships

Layers of Architecture

We will design the **architecture layers** of the system in a structured way, ensuring **separation of concerns** and **modularity**. The system will be organized into the following layers:

Client/UI → Controller Layer → Service Layer → Domain Layer

CONTROLLERS:

1. WalletController

- - Wallet createWallet(int userId)
- - long getBalance(String accountNumber)
- - AccountStatement getStatement(String accountNumber, LocalDateTime start, LocalDateTime end)

2. TransactionController

- - Transaction transfer(TransactionRequest request)
- - Transaction initiateDeposit(AddMoneyRequest request)
- - void handlePaymentCallback(String providerRef, TransactionStatus status)
- - Transaction withdraw(String accountNumber, long amount, String description)

3. AdminController

- - void suspendWallet(String accountNumber)
 - - void closeWallet(String accountNumber)
 - - void reopenWallet(String accountNumber)
-

SERVICES:

1. WalletService

- - Wallet createWallet(int userId)
- - Wallet getByAccountNumber(String accountNumber)
- - boolean isActive(String accountNumber)

2. TransactionService

- - Transaction transfer(TransactionRequest request)
- - Transaction initiateDeposit(AddMoneyRequest request)
- - void handleDepositCallback(String providerRef, TransactionStatus status)
- - Transaction withdraw(String accountNumber, long amount, String description)
- - AccountStatement getStatement(String accountNumber, LocalDateTime start, LocalDateTime end)

3. PaymentGatewayService (Strategy)

- - String initiatePayment(String accountNumber, long amount, String paymentMethod, String paymentGateway, Map<String, String> paymentDetails)
- - boolean verifyCallback(String providerRef, TransactionStatus status)

4. LockService (Distributed)

- - boolean acquire(String key, long timeoutMs)
- - void release(String key)

5. NotificationRouter (Simple registry + dispatch)

- - void register(String channelName, NotificationChannel channel)
 - - void send(String channelName, NotificationMessage message)
-

PAYMENT GATEWAY STRATEGY (Scalable Providers):

- **PaymentGatewayProvider (Strategy Interface)**
 - + String getName()
 - + String initiatePayment(String accountNumber, long amount, String paymentMethod, Map<String, String> paymentDetails)
 - + boolean verifyCallback(String providerRef, TransactionStatus status)
 - StripePaymentGatewayProvider (Concrete)
 - RazorpayPaymentGatewayProvider (Concrete)
 - PayPalPaymentGatewayProvider (Concrete)
 - MockPaymentGatewayProvider (Concrete - simulation)
 - **PaymentGatewayRouter (Context/Router)**
 - - Map<String, PaymentGatewayProvider> providers
 - + String selectProvider(String preferredGateway, long amount, String currency)
 - + PaymentGatewayProvider resolve(String gatewayName)
-

NOTIFICATION STRATEGY:

- **NotificationChannel (Interface)**
 - - void send(NotificationMessage message)
- **NotificationRouter (Registry + Single-dispatch)**
 - - Map<String, NotificationChannel> channels
 - + void register(String channelName, NotificationChannel channel)
 - + void send(String channelName, NotificationMessage message)
- EmailNotificationChannel
- SmsNotificationChannel
- NotificationMessage (to, subject, body)

REPOSITORIES:

1. WalletRepository

- - Optional<Wallet> findByAccountNumber(String accountNumber)
- - Wallet save(Wallet wallet)
- - void updateBalance(int walletId, long deltaMinor)

2. UserRepository

- - Optional<User> findById(int userId)

3. TransactionRepository

- - Transaction save(Transaction transaction)
- - List<Transaction> findByWalletAndRange(int walletId, LocalDateTime start, LocalDateTime end)
- - Optional<Transaction> findById(String transactionId)

PATTERNS & KEY RELATIONSHIPS:

- **Association (uses)** – Transaction references Wallet via fromWalletId/toWalletId
 - **Association (manages)** – Wallet is one-to-one with User
 - **Dependency** – Services depend on repositories, LockService, PaymentGatewayService
 - **Strategy Pattern** – Payment gateway providers are interchangeable
 - **Repository Pattern** – Database access abstraction
 - **Event-driven Notifications** – Transaction completion triggers notifications
-

Step 5: Core Use Cases & Methods

1. Transfer Use Case:

transfer(request) →

- Validate: non-negative amount, amount ≥ 1 (minor unit), not self-transfer
 - Fetch wallets by account numbers; ensure both ACTIVE and not SUSPENDED/CLOSED
 - Acquire distributed locks on both wallet IDs in sorted order
 - Within DB transaction:
 - Ensure fromWallet.balance $\geq$ amount
 - Update fromWallet.balance -= amount
 - Update toWallet.balance += amount
 - Create Transaction(COMPLETED, type=TRANSFER)
 - Release locks; send notification via NotificationRouter
-

2. Deposit Use Case:

initiateDeposit(request) →

- Validate: amount $\geq$ minimum, wallet ACTIVE
- Create Transaction(PENDING, type=DEPOSIT, paymentGateway, paymentMethod)
- Select provider using PaymentGatewayRouter
- Call provider.initiatePayment(...) → return providerRef

handleDepositCallback(providerRef, status) →

- Verify callback authenticity
- If SUCCESS and not already processed:

- Credit wallet (inside DB transaction)
 - Mark Transaction COMPLETED
 - Send notification via NotificationRouter
 - If FAILURE/TIMEOUT: mark Transaction FAILED (idempotent)
-

3. Withdraw Use Case (Optional Stub):

withdraw(accountNumber, amount) →

- Validate KYC/limits and wallet ACTIVE
 - Create Transaction(PENDING, type=WITHDRAWAL)
 - External payout provider integration (out of scope)
 - On success: debit wallet and mark COMPLETED; else FAILED
-

4. Account Statement Use Case:

getStatement(accountNumber, start, end) →

- Lookup wallet
 - Query TransactionRepository.findByWalletAndRange(...)
 - Build AccountStatement with current balance + transactions
 - Return DTO (no persistence)
-

Step 6: Apply OOP Principles & Design Patterns

Design Patterns Used:

- **Repository Pattern** – data access abstraction for Wallet/User/Transaction

- **Service Layer** – business logic separation (WalletService/TransactionService)
 - **Strategy Pattern** – PaymentGatewayProvider + Router; NotificationChannel + Router
 - **Domain Events** – transaction completion triggers notifications (or direct router call)
 - **RESTful API Design** – clean resource-oriented endpoints
-

OOP Principles Applied:

- **Single Responsibility** – each service focuses on one concern
 - **Open/Closed** – add new payment gateways or notification channels without modifying core logic
 - **Encapsulation** – wallet balance updates only through TransactionService
 - **Dependency Inversion** – services depend on interfaces (repositories, gateway, notification, lock)
 - **Idempotency** – callbacks/requests can be safely retried without double credit/debit
-

Step 7: Handle Edge Cases

Edge Case Solutions:

- **Insufficient Funds:**
 - Validate within transaction
 - Reject transfer/withdraw with clear error
- **Self-Transfer:**
 - Disallow when fromAccountNumber == toAccountNumber
- **Negative/Zero Amount:**

- Reject request
 - Minimum transferable/depositable amount = 1 minor unit (e.g., 1 cent)
 - **Closed/Suspended Wallet:**
 - Block transfers, deposits, withdrawals
 - Allow statement retrieval only
 - **Duplicate Client Requests (Retries):**
 - Use client-provided idempotency key mapped to Transaction
 - Return prior result if request is repeated
 - **Duplicate Payment Callbacks:**
 - Ensure callback handler is idempotent
 - Update only if status changes from PENDING → COMPLETED/FAILED
 - **Concurrency on Same Wallet:**
 - Use distributed locks (STEP-7.5)
 - Use DB transaction for atomic balance updates
 - **Rounding/Precision:**
 - All balances stored in minor units (long)
 - No floating-point arithmetic
 - **Large Amounts / Overflow:**
 - Validate amount and balances do not overflow long
 - Define maximum allowed per transfer
 - **Transfers to Non-Existent Wallet:**
 - Validate destination wallet exists and is ACTIVE before debiting source
 - **Statement Pagination:**
 - Support pagination and time-window filtering
 - Prevents extremely large payloads
 - **Notification Failures:**
 - Log failures and retry asynchronously
 - Transaction commit is never blocked by notification issues
-

Step 7.5: Concurrency Handling Methods (Preventing Double Deduction)

Overview:

Multiple concurrent requests trying to transfer from the same wallet can cause double deduction or race conditions. Here are 3 generic approaches to handle this:

Approach 1: Database Transaction with Row-level Locking

Description:

- Use database transaction with high isolation level (SERIALIZABLE or REPEATABLE_READ)
- Acquire exclusive locks on wallet rows using SELECT FOR UPDATE
- Locks are held until transaction commits/rolls back

Example Flow:

- BEGIN TRANSACTION
- SELECT balance FROM wallet WHERE id = 1 FOR UPDATE (locks row)
- SELECT balance FROM wallet WHERE id = 2 FOR UPDATE (locks row)
- Validate: wallet1.balance >= amount
- UPDATE wallet SET balance = balance - amount WHERE id = 1
- UPDATE wallet SET balance = balance + amount WHERE id = 2
- INSERT INTO transaction (...) VALUES (...)
- COMMIT TRANSACTION

Benefits:

- Database ensures atomicity (all or nothing)
- Prevents concurrent modifications
- Works with any SQL database (PostgreSQL, MySQL, Oracle, etc.)
- Language-independent

Limitations:

- Locks held for entire transaction duration
 - Can cause deadlocks if locks are acquired in different order
 - May reduce throughput under high concurrency
-

Approach 2: Optimistic Locking with Version Field

Description:

- Add version/timestamp field to Wallet entity
- Read wallet with current version
- Before update, verify version hasn't changed
- If version changed, transaction fails (retry or reject)

Example Flow:

- READ wallet (id=1, balance=100, version=5)
- Validate: balance >= amount
- UPDATE wallet SET balance = balance - amount, version = version + 1 WHERE id = 1 AND version = 5
- If update affects 0 rows → Version changed → Retry or fail
- If update affects 1 row → Success → Continue with toWallet update

Benefits:

- No long-held locks
- Better for high-read scenarios
- Works with any database and language
- Retry mechanism handles conflicts

Limitations:

- Requires retry logic
 - Can fail multiple times before success
 - More complex error handling
-

Approach 3: Distributed Locking (Selected Approach)

Description:

- Use distributed lock manager (Redis, Memcached, etc.)
- Acquire lock on wallet ID before transaction
- Release lock after transaction completes
- Set lock expiration to prevent deadlocks

Example Flow:

- ACQUIRE LOCK("wallet_1", timeout=5 seconds)
 - If lock acquired → Proceed
 - If lock exists → Wait (block) until lock is released or timeout expires
 - If timeout expires → Return error
- READ wallet balance
- Validate: balance >= amount
- UPDATE wallet balances
- CREATE transaction record
- RELEASE LOCK("wallet_1")

Benefits:

- Works across multiple application servers (distributed systems)
- Language and database agnostic
- Prevents concurrent access at application level
- Lock expiration prevents deadlocks
- Handles high concurrency with proper lock management

Limitations:

- Requires external lock service (Redis, etc.)
- Network latency for lock operations
- Need to handle lock expiration and renewal

Selected Approach: Distributed Locking (Approach 3)

The system uses **distributed locking** for the following reasons:

- **Scalability:** Works across multiple application servers
- **Flexibility:** Language and database agnostic
- **Reliability:** Lock expiration prevents deadlocks
- **Performance:** Efficient for high-concurrency scenarios
- **Interview-friendly:** Demonstrates understanding of distributed systems

Implementation Details:

- **Lock Service:** Redis or similar distributed cache
 - **Lock Key Format:** "wallet_lock_{walletId}"
 - **Lock Timeout:** 5 seconds (prevents deadlocks)
 - **Lock Acquisition:** Wait with timeout behavior – transaction blocks (waits) until lock is available or timeout expires
 - **Lock Behavior:** If lock is taken, transaction waits (retries) up to timeout period; if timeout expires, transaction fails
 - **Lock Ordering:** Acquire locks in sorted order (wallet IDs) to prevent deadlocks
 - **Lock Release:** Always release in finally block to prevent lock leaks
-

Step 9: Future Improvements

Future Functional Improvements:

- Multi-currency wallets with FX conversion and per-currency limits
- KYC/AML workflows, risk scoring, and fraud detection rules
- Peer-to-merchant (P2M) payments with QR codes and payment links
- Scheduled/recurring transfers and standing instructions
- Dispute management and chargeback handling

- Advanced notifications (push/SMS) and webhook subscriptions
 - Exportable statements (CSV/PDF) with filters and receipt IDs
 - Integration with bank rails for deposits/withdrawals
-

Summary

This design targets a 60–120 minute interview with 5 core flows:

- **Wallet Creation:** POST /api/wallets (one wallet per user)
 - **Deposit (Two-Phase):** Initiate + Callback via PaymentGateway
 - **Transfer:** Locked, atomic balance movement with audit trail
 - **Withdraw (Stub):** Illustrates payout workflow and statuses
 - **Account Statement:** Time-windowed transaction history DTO
-

Highlights:

- Minor units for money; atomic updates under distributed locks (STEP-7.5)
- Repository/Service layering; Strategy Pattern for payment gateways
- Idempotent callbacks and client request handling
- Clear edge-case coverage suitable for production discussions